

סנכרון

מערכות הפעלה
לינוקס

The problem model

- Multiple processes
- Processes can apply read, write, or stronger operations to shared memory
- Completely asynchronous
- We assume processes never fail-stop (next step eventually scheduled)
- Processes never stuck in critical section (CS)
- Process may try to capture lock any number of times (0 or more)

Correctness and progress conditions

1. *Mutual Exclusion* – No two processes are in the *critical section* (CS) at the same time
2. *Deadlock Freedom* – If processes are trying to enter the CS *one* will eventually enter it
3. *Starvation Freedom* – When a process tries to enter its CS, it will eventually succeed

Peterson's Solution

```
int interested[2];  
int barrier;  
interested[0]=interested[1]=FALSE
```

Process = 0

```
void enter_region(...){  
    int other = 1;  
    interested[0] = TRUE;  
    barrier = 0;  
    while (barrier == 0 &&  
           interested[other] == TRUE);  
}  
// critical section  
void leave_region(...){  
    interested[0] = FALSE;}
```

Process = 1

```
void enter_region(...){  
    int other = 0;  
    interested[1] = TRUE;  
    barrier = 1;  
    while (barrier == 1 &&  
           interested[other] == TRUE);  
}  
// critical section  
void leave_region(...){  
    interested[1] = FALSE;}
```

Peterson's Solution

```
N=2; interested[0]=interested[1]=FALSE;
int barrier;
int interested[N];
void enter_region(int process){
    int other = 1- process;
    barrier = process;
    interested[process] = TRUE; /* signal that you're interested */
    while (barrier == process &&
           interested[other] == TRUE); /* null statement */
}
void leave_region(int process){
    interested[process] = FALSE; /* departure from critical region */
}
```

Consider the following variant of Peterson's solution where the two red lines are swapped. Does this variant solve the mutual exclusion problem?

Peterson's Solution

No.

Consider, for example, the following scenario:

Process 1 runs: sets barrier 1 and stops.

Now, process 0 runs: sets barrier 0, sets interested[0] True, enters to the critical section (because interested[1] is False), and stops.

Now, process 1 runs: sets interested[1] True, and enters the critical section (because barrier is 0).

Peterson's Solution

Process = 0

Process = 1

```
int barrier;  
int interested[2];  
interested[0]=interested[1]=FALSE
```

```
void enter_region(...){  
    int other = 1; Step 3  
    barrier = 0; Step 4  
    interested[0] = TRUE; Step 5  
    while (barrier == 0 && Step 6  
           interested[other] == TRUE);  
}  
    Passed the while loop (interested[1]==FALSE), now  
    inside CS INTERRUPTED! Moving to P1  
// critical section  
void leave_region(...){  
    interested[0] = FALSE;}
```

```
void enter_region(...){  
    int other = 0; Step 1  
    barrier = 1; Step 2 INTERRUPTED! Moving to P0  
    interested[1] = TRUE; Step 7  
    while (barrier == 1 && Step 8  
           interested[other] == TRUE);  
}  
    Passed the while loop (since barrier==0), now  
    inside CS  
// critical section  
void leave_region(...){  
    interested[1] = FALSE;}
```

Peterson's Solution

- Assume the while statement in Peterson's solution is changed to:
*while (barrier != process &&
interested[other] == TRUE)*
- **Describe** a scheduling scenario in which we will receive a **mutual exclusion violation** and one in which we will **NOT** receive a Mutual Exclusion violation

Peterson's Solution

Process = 0

Process = 1

```
interested[0]=interested[1]=FALSE  
int barrier;  
int interested[2];
```

```
void enter_region(...){  
    int other = 1;  
    interested[0] = TRUE;  
    barrier = 0;  
    while (barrier != 0 &&  
           interested[other] == TRUE);  
}  
// critical section  
void leave_region(...){  
    interested[0] = FALSE;
```

```
void enter_region(...){  
    int other = 0;  
    interested[1] = TRUE;  
    barrier = 1;  
    while (barrier != 1 &&  
           interested[other] == TRUE);  
}  
// critical section  
void leave_region(...){  
    interested[1] = FALSE;
```

Peterson's Solution

No mutual exclusion violation:

1. One enters and exits and only afterwards the second receives a time quanta.
2. P0 – interested[0]=true,
P0 – barrier = 0,
P1- interested[1]=true,
P1 – barrier =1,
P1 passes while loop and enters CS,
P0 – stop in while loop.
3. Many more....

Peterson's Solution

Mutual exclusion violation scenario

Process = 0

Process = 1

interested[0]=interested[1]=FALSE
int barrier;
int interested[2];

```
void enter_region(...){  
    int other = 1; Step 1  
    interested[0] = TRUE; Step 2  
    barrier = 0; Step 3  
    while (barrier != 0 && Step 4  
           interested[1] == TRUE);  
}  
    Passed the while loop (barrier==0),  
    now inside CS INTERRUPTED! Moving to P1  
//critical section  
void leave_region(...){  
    interested[0] = FALSE;}
```

```
void enter_region(...){  
    int other = 0; Step 5  
    interested[1] = TRUE; Step 6  
    barrier = 1; Step 7  
    while (barrier != 1 && Step 8  
           interested[0] == TRUE);  
}  
    Passed the while loop (barrier==1),  
    now inside CS  
//critical section  
void leave_region(...){  
    interested[1] = FALSE;}
```

Synchronization Algorithm

ב. (13 נק') להלן אלגוריתם למניעה הדדית.

```
shared int turn
shared boolean busy initially false
Program for process  $i \in (1, \dots, n)$ 
1 do {
2   do {
3     turn := i
4   } while (busy)
5   busy := true
6 } while (turn != i)
7 Critical section
8 busy := false
```

i. (6 נק') האם האלגוריתם מקיים מניעה הדדית? אם כן, נמקו בקצרה. אם לא, כתבו תסריט מדויק המדגים כי יש הפרה של מניעה הדדית.

כן, כדי שתהליך יכנס הוא צריך לבצע את שורות 3 עד 6 (כולל) ברצף ללא הפרעה. אם 2 יצאו מהלולאה הפנימית לפני שאחר מהם ביצע את 5 רק זה ששינה אחרון את התור יכנס.

ii. (7 נק') האם האלגוריתם מקיים חופש מקיפאון? אם כן, נמקו בקצרה. אם לא, כתבו תסריט מדויק המדגים כי אין חופש מקיפאון.

לא, נניח $n=2$. תהליך 1 מבצע עד שורה 5 כולל, תהליך 2 מבצע עד שורה 3 כולל. תהליך 2 כרגע תקובע בלולאה הפנימית, תהליך 1 ממשיך ורואה ש $turn \neq 1$ (כי הוא שווה ל-2) וחוזר ללולאה הפנימית בה הוא תקוע כי $busy = true$ ואף תהליך לא ישחרר אותו.

Starvation-free mutex using test-and-set

Consider the following simple algorithm

```
1  int lock initially 0
2  boolean interested[n] initially {false,...,false}

Code for process  $i \in \{0, \dots, n-1\}$ 
3  interested[i]:=true
4  await ((test-and-set(lock)=0) OR (interested[i] =false))
5  Critical Section
6  j:=(i+1) modulo n
7  while (j  $\neq$  i) AND (interested[j] = false)
8      j:=(j+1) modulo n
9  if (j=i)
10     lock:=0
11 else
12     interested[j]:=false
13 interested[i]=false
```

Test-and-set(w)
do atomically
 prev:=w
 w:=1
 return prev

Is this algorithm deadlock free? Prove or disprove by providing an accurate scenario which leads to a deadlock.

Starvation-free mutex using test-and-set

```
1  int lock initially 0
2  boolean interested[n] initially {false,...,false}

Code for process  $i \in \{0, \dots, n-1\}$ 
3  interested[i]:=true
4  await ((test-and-set(lock)=0) OR (interested[i] =false))
5  Critical Section
6  j:=(i+1) modulo n
7  while (j  $\neq$  i) AND (interested[j] = false)
8      j:=(j+1) modulo n
9  if (j=i)
10     lock:=0
11 else
12     interested[j]:=false
13 interested[i]=false
```

Step 1 – P0 runs until beginning of 13

Interrupt! Moving to P1..

Step 2 – P1 runs all the way through the CS, finds process 0 since **interested[0] = true.**

Step 3 – As a result, **P1 resets interested[0] to false.**

Step 4 – P1 continue and finish its run.

Interrupt! Moving to P0..

Step 5 – P0 runs line 13 and finish its run.

After Step 5, the system is in DEADLOCK state, since the lock is occupied (lock=1 since P1 executed the 'else' statement) and the rest of the processes that will later come will stuck in line 4.

Counting Semaphores

- An interface of atomic functions supplying a simple synchronization tool
- Programmers don't need to bother with synchronization algorithms

Counting semaphore

Init(i)

S = i

down(S) [the 'p' operation]

if ($S \leq 0$)

the process is blocked.

It will resume execution only
after it is woken-up

else

S--

up(S) [the 'v' operation]

if (there are blocked processes)

wake-up one of them

else

S++

- Semaphore's interface doesn't enforce the implementation of *starvation freedom*.
- All operations are ATOMIC! That is, up(s) is more than just $s := s + 1$
- There is no way to access the semaphore's internal value. Any attempt to access it is a mistake.
- Always remember to initialize the semaphore

שיתוף זיכרון

שיתוף זיכרון

כדי להבין איך יכול להיות מצב שבו שני Processים ניגשים לאותו מקום ב-RAM יש צורך להבין מהו מיפוי זיכרון. לכן חלק ניכר נקדיש למנגנון מיפוי הזיכרון, ולאחר מכן המשיך ההסבר על שיתוף הזיכרון יהיה פשוט.

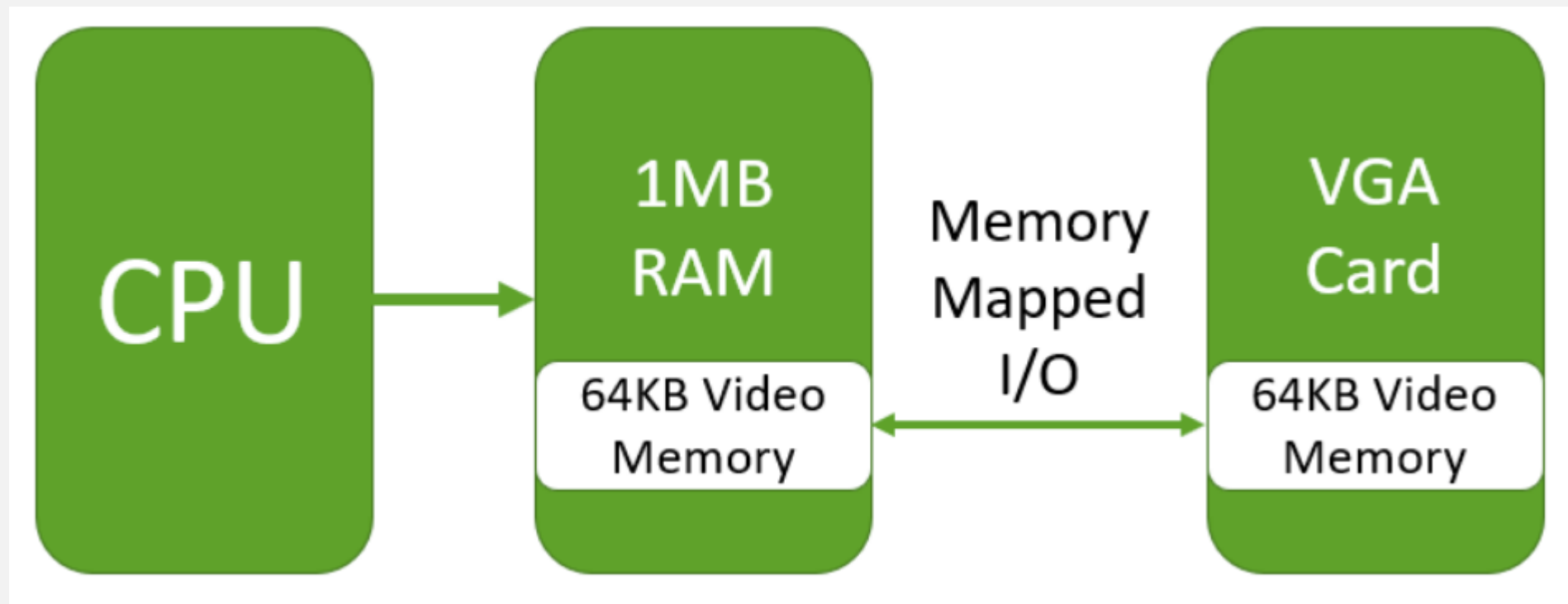
מיפוי זיכרון הוא תהליך שבו נוצר קישור בין זיכרון פיזי ב-RAM לבין זיכרון של התקן חומרה כלשהו. התקן חומרה שבוצע עבורו מיפוי זיכרון ל-RAM נקרא Memory Mapped I/O, כאשר ה-I/O הם קיצור של Input/Output. מה המשמעות של קישור בין שני זיכרונות? הכוונה ליצירת מנגנון שבו כל שינוי בזיכרון אחד יוצר את אותו השינוי בזיכרון אחר. היתרון בשיטה הזו היא שהיא מאפשרת למעבד לפנות בקלות להתקני חומרה, בלי צורך להפעיל Interruptים (פסיקות חומרה) ובלי צורך בפניה ל-Portים ושימוש בפקודות מיוחדות.

מבין התקני החומרה השונים שניתן לבצע אליהם מיפוי זיכרון, התקן חומרה ספציפי הוא התקן לשמירת קבצים כגון הדיסק הקשיח. אם המיפוי מבוצע לקובץ ולא למקרה הכללי יותר של התקן חומרה, הקובץ נקרא פשוט Memory Mapped File.

מיפוי זיכרון קלט/פלט - Memory Mapped I/O

השימוש ב-Memory Mapped I/O קלאסי לכרטיסי גרפיקה, עקב מהירות העדכון הרבה שהם דורשים, לדוגמה במשחקי מחשב, מיפוי זיכרון הוא אפשרות מקובלת. ניקח מערכת שיש בה מעבד, זיכרון RAM וכרטיס גרפי. כל מה שמוצג למסך המשתמש חייב להיות בזיכרון של הכרטיס הגרפי. נמחיש את העקרון באמצעות מעבד ה-8086 ההיסטורי, אך העקרון תקף גם כיום.

למעבד ה-8086 היתה גישה ל-MB1 של זיכרון RAM. לצד המעבד היה מותקן כרטיס גרפי VGA שמסוגל להציג 256 צבעים על מסך שגודלו 320X200 פיקסלים. כל פיקסל דרש בית אחד, לכן גודלו של הזיכרון של כרטיס ה-VGA היה 64KB. כל הזיכרון הגרפי מופה אל אזור מסוים בזיכרון ה-RAM, שהתחיל בכתובת 0xA0000 וגודלו היה 0x10000 (כלומר 64KB).

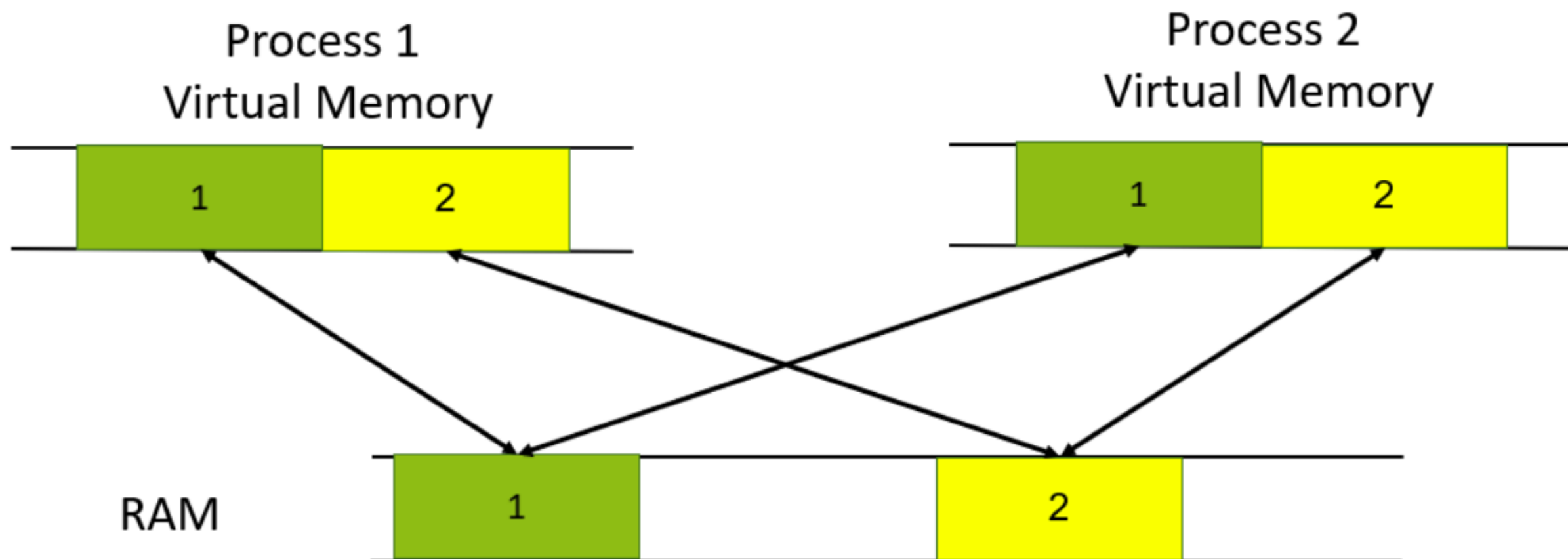


היתרון של מיפוי הזיכרון הוא הפשטות והמהירות: כדי לפנות לזיכרון הוידאו, המעבד פשוט מבצע פקודות העתקה אל אזור הזיכרון שממופה אצלו ל-RAM. אין צורך בפסיקות חומרה איטיות. החיסרון- המיפוי גוזל חלק מזיכרון ה-RAM של המעבד, שלמעשה הפך להיות בלתי שמיש לכל מטרה שאינה תקשורת עם כרטיס הוידאו.

זיכרון משותף - Shared Memory

מיפוי זיכרון יכול להפוך בקלות לשיתוף זיכרון. במצב של שיתוף זיכרון, אותו אזור ב-RAM ממופה לתוך הזיכרון הוירטואלי של שני Processים.

האיור הבא ממחיש את שיתוף הזיכרון, שני קטעי זיכרון ב-RAM ממופים אל שני Process-ים. במצב זה, הזיכרונות של שני ה-Process-ים קשורים ביניהם באותם קטעי זיכרון ב-RAM. אם אחד מה-Process-ים ישנה את הזיכרון בקטעים המשותפים, ה-Process השני יראה את השינוי.



כדי להשיג Shared Memory, יש לפעול לפי השלבים הבאים:

:1 Process

1. פותח קובץ - CreateFile

2. יוצר מיפוי בין הקובץ לזיכרון – CreateFileMapping. כחלק מתהליך המיפוי, נותן שם לאזור בזיכרון. היזכרו בפרמטר האחרון בפונקציה CreateFileMapping, שבדוגמת הקוד נקרא "MyFile"

3. משתמש במיפוי עבור חלק מהקובץ - MapViewOfFile

:2 Process

1. לוקח גישה לאזור הממופה – OpenFileMapping. כדי להשיג גישה, צריך להכיר את שם האזור (בדוגמה - "MyFile")

2. משתמש במיפוי עבור חלק מהקובץ – MapViewOfFile

לקריאה נוספת, מתוך הדרכה של מיקרוסופט:

[https://docs.microsoft.com/en-us/previous-versions/ms810613\(v=msdn.10\)?redirectedfrom=MSDN](https://docs.microsoft.com/en-us/previous-versions/ms810613(v=msdn.10)?redirectedfrom=MSDN)

דוגמאות ל-Shared Memory

נסקור שתי דוגמאות קלאסיות לשיתוף זיכרון: ה-Kernel ו-DLLים.

כפי שלמדנו, חלק ממרחב הזיכרון של כל Process מוקדש לקוד של ה-Kernel של מערכת ההפעלה. דבר זה מאפשר ל-Process לפנות לפונקציות של מערכת ההפעלה.

מה קורה כאשר מבוצע Context Switch אל Process חדש? כיוון שגם ה-Process החדש צריך את אותו הקוד של מערכת ההפעלה במרחב הזיכרון, זה יהיה בזבוז עצום להחליף את כל הדפים ב-RAM שיש בהם את הקוד של מערכת ההפעלה. הרבה יותר פשוט לבצע מיפוי של קוד מערכת ההפעלה שנמצא כבר ב-RAM אל הזיכרון של ה-Process החדש, ובאופן כללי אל כל ה-Processes שרצים. התוצאה היא שקוד של מערכת ההפעלה נמצא Shared Memory בין כל ה-Processes.

מה קורה כאשר תוכניות רבות מייבאות את אותה חבילת קוד? אין הגיון בכך שכל Process יעתיק את חבילת הקוד לתוך ה-RAM.